

Báo Cáo Đồ Án: Phần 1

Tên: Lê Phạm Đăng Khiêm

MSSV: 24120341

Môn: Toán ứng dụng và thống kê

Mục lục

1	Giới Thiệu Đồ Án	3
2	Tổng Quan Cấu Trúc Đồ Án	3
2.1	Danh Sách File	3
3	Thuật Toán Khử Gauss (gaussian.py)	3
3.1	Mô Tả Chung	3
3.1.1	Các Hàm Hỗ Trợ	3
3.1.2	Khử Ma Trận về Bậc Thang	4
3.1.3	Tìm Nghiệm Hệ Phương Trình	4
3.1.4	Cách Sử Dụng	5
3.1.5	Ví Dụ Đầu Ra	5
4	Tính Định Thức (determinant.py)	6
4.1	Thuật Toán	6
4.2	Cách Sử Dụng	6
4.3	Các Trường Hợp Kiểm Thử	6
5	Tính Ma Trận Nghịch Đảo (inverse.py)	6
5.1	Thuật Toán Gauss-Jordan	6
5.2	Cách Sử Dụng	7
5.3	Ứng Dụng Kiểm Thử	7
6	Tính Hạng và Cơ Sở (rank_basis.py)	7
6.1	Định Nghĩa	7
6.2	Thuật Toán	7
6.3	Cách Sử Dụng	7
7	Kiểm Chứng và Xác Nhận (part1_demo.ipynb)	8
7.1	Mục Đích	8
7.2	Nội Dung Chính	8
7.2.1	Test Gaussian Elimination	8
7.2.2	Test Back Substitution	8
7.2.3	Test Determinant	8
7.2.4	Test Matrix Inverse	8
7.2.5	Test Rank and Basis	9

7.3	Kiểm Chứng bằng NumPy và SymPy	9
8	Hướng Dẫn Sử Dụng (README.md)	10
8.1	Giải Thích Hàm gaussian_eliminate	10
8.1.1	Ý Nghĩa của Nghiệm Hệ	10
8.1.2	Ví Dụ Kiểm Tra Nhanh	11
8.2	Xử Lý Sai Số Số Thực	11
8.3	Phân Biệt Hàm	11
9	Ưu Điểm và Đặc Điểm Của Đồ Án	11
9.1	Hiệu Năng Và Độ Ổn Định	11
9.2	Partial Pivoting	11
9.3	Xử Lý Nhiều Trường Hợp	11
9.4	Hiển Thị Rõ Ràng	12
10	Các Thuật Toán Không Được Sử Dụng	12
11	Kết Luận	12

1 Giới Thiệu Đồ Án

Hỗ trợ Github copilot: Hỗ trợ viết báo cáo bằng latex nhanh chóng, kiểm chứng bằng numpy/sympy để giải thích kết quả trả về.

Đồ án triển khai các thuật toán đại số tuyến tính cơ bản bằng Python, gồm:

- Khử Gauss (Gaussian Elimination)
- Tính định thức (Determinant Calculation)
- Tính ma trận nghịch đảo (Matrix Inverse)
- Tính hạng ma trận và cơ sở không gian (Rank and Basis Calculation)
- Các hàm hỗ trợ và định dạng hiển thị

Toàn bộ phép tính trong phần cài đặt sử dụng số thực `float`. Để ổn định số học, chương trình dùng ngưỡng $\text{EPS} = 1\text{e-}12$ khi so sánh với 0.

2 Tổng Quan Cấu Trúc Đồ Án

2.1 Danh Sách File

- `gaussian.py` - Hàm khử Gauss, thay thế ngược và các hàm hỗ trợ
- `determinant.py` - Tính định thức bằng khử Gauss
- `inverse.py` - Tính ma trận nghịch đảo bằng Gauss-Jordan
- `rank_basis.py` - Tính hạng và cơ sở không gian
- `part1_demo.ipynb` - Notebook minh họa với các bài kiểm thử
- `README.md` - Hướng dẫn sử dụng
- `report/` - Thư mục báo cáo
 - `report/report.tex` - File mã nguồn LaTeX của báo cáo
 - `report/report.pdf` - File PDF báo cáo

3 Thuật Toán Khử Gauss (`gaussian.py`)

3.1 Mô Tả Chung

Tệp `gaussian.py` chứa các hàm cốt lõi cho thuật toán khử Gauss:

3.1.1 Các Hàm Hỗ Trợ

`to_float(value)` Chuyển đổi giá trị đầu vào về kiểu `float`:

```
1 def to_float(value):
2     return float(value)
```

định_dạng_hiển_thị(value) Định dạng số thực để hiển thị gọn (làm sạch các giá trị rất nhỏ về 0 và hiển thị số nguyên khi phù hợp).

in_ma_tran_dep(ma_tran) In ma trận dưới dạng cấu trúc bảng gọn gàng, căn lề phải

3.1.2 Khử Ma Trận về Bậc Thang

Hàm `khu_ma_tran_ve_bac_thang(ma_tran_chuan_hoa, b_chuan_hoa, eps)` thực hiện:

1. Duyệt qua từng cột từ trái sang phải
2. Tìm **pivot** - phần tử có giá trị tuyệt đối lớn nhất trong cột (Partial Pivoting)
3. Hoán đổi hàng nếu cần để đưa pivot lên vị trí hiện tại
4. Khử các phần tử dưới pivot bằng phép biến đổi hàng:

$$\text{Hàng } r := \text{Hàng } r - \frac{a_{r,c}}{a_{k,c}} \times \text{Hàng } k$$

Trong quá trình khử, các so sánh với 0 được thực hiện theo ngưỡng EPS thay vì so sánh bằng tuyệt đối để giảm ảnh hưởng sai số dấu phẩy động. Trả về:

- `ma_tran_sau_khu`: Ma trận dạng bậc thang
- `b_sau_khu`: Vector b sau khi áp dụng các phép biến đổi tương ứng
- `so_lan_doi_hang`: Số lần hoán đổi hàng
- `pivot_rows`: Danh sách chỉ số hàng chứa pivot
- `pivot_cols`: Danh sách chỉ số cột chứa pivot

3.1.3 Tìm Nghiệm Hệ Phương Trình

Hàm `tim_nghiem_he` xác định loại nghiệm:

1. **Hệ vô nghiệm**: Khi tồn tại hàng mà tất cả phần tử bên trái = 0 nhưng phần tử bên phải $\neq 0$
2. **Hệ có vô số nghiệm**: Khi số pivot < số cột (có biến tự do)
3. **Hệ có nghiệm duy nhất**: Khi số pivot = số cột

Với hệ có vô số nghiệm, hàm `nghiem_tong_quat_he_vo_so_nghiem` tính:

- Nghiệm riêng $\mathbf{x}_0$
- Các vector cơ sở tương ứng với các biến tự do
- Biểu thức tham số cho mỗi ẩn

3.1.4 Cách Sử Dụng

gaussian_eliminate(A, b) Giải hệ phương trình tuyến tính $A\mathbf{x} = \mathbf{b}$:

```

1 A = [[1, 2, 0, 2], [3, 5, -1, 6], [2, 4, 1, 2], [2, 0, -7, 11]]
2 b = [6, 17, 12, 7]
3 ma_tran_sau_khu, nghiem_he, so_lan_doi_hang = gaussian_eliminate(A, b)
4
5 if nghiem_he is None:
6     print("Vo nghiem")
7 elif len(nghiem_he) > 0 and isinstance(nghiem_he[0], str):
8     print("Vo so nghiem:", nghiem_he)
9 else:
10    print("Nghiem duy nhat:", nghiem_he)

```

3.1.5 Ví Dụ Đầu Ra

Trường hợp 1: Hệ có nghiệm duy nhất

```

1 A = [[2, 1, -1], [-3, -1, 2], [-2, 1, 2]]
2 b = [8, -11, -3]
3 ma_tran_sau_khu, nghiem_he, so_lan_doi_hang = gaussian_eliminate(A, b)
4 print("Nghiem duy nhat:", nghiem_he)
5 # Output: Nghiem duy nhat: [2.0, 3.0, -1.0]

```

Trường hợp 2: Hệ vô số nghiệm

```

1 A = [[1, 2, 3], [2, 4, 6]]
2 b = [5, 10]
3 ma_tran_sau_khu, nghiem_he, so_lan_doi_hang = gaussian_eliminate(A, b)
4 print("Vo so nghiem:", nghiem_he)
5 # Output: Vo so nghiem: ['5.0 - 2.0*t_0 - 3.0*t_1', '0.0 - 0.0*t_0 +
    1.0*t_1', 't_0']

```

Trường hợp 3: Hệ vô nghiệm

```

1 A = [[1, 2, 3], [2, 4, 6], [1, 0, 1]]
2 b = [5, 9, 2]
3 ma_tran_sau_khu, nghiem_he, so_lan_doi_hang = gaussian_eliminate(A, b)
4 if nghiem_he is None:
5     print("Vo nghiem")
6 # Output: Vo nghiem

```

print_gaussian_eliminate(A, b) In kết quả định dạng đẹp ra màn hình

back_substitution(U, c) Giải hệ tam giác trên $U\mathbf{x} = \mathbf{c}$ bằng thay thế ngược:

$$x_i = \frac{c_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}}$$

4 Tính Định Thức (determinant.py)

4.1 Thuật Toán

Định thức được tính thông qua khử Gauss:

$$\det(A) = (-1)^s \times \prod_{i=1}^n a_{ii}$$

trong đó:

- a_{ii} là các phần tử trên đường chéo của ma trận dạng bậc thang
- s là số lần hoán đổi hàng

Chú ý: Nếu tồn tại hàng hoàn toàn bằng 0, định thức = 0.

4.2 Cách Sử Dụng

```
1 A = [[2, 3], [1, 5]]
2 det = determinant(A) # Tra về float
3 print(f"Định thức: {det}")
4
5 # Hoac in theo định dạng đẹp
6 print_determinant(A)
```

4.3 Các Trường Hợp Kiểm Thử

- Ma trận 2×2 : $\det \begin{pmatrix} 2 & 3 \\ 1 & 5 \end{pmatrix} = 7$
- Ma trận 3×3 : $\det \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix} = -49$
- Ma trận phụ thuộc tuyến tính: $\det \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 2 \end{pmatrix} = 0$
- Ma trận đơn vị: $\det(I) = 1$
- Ma trận đường chéo: $\det =$ tích các phần tử trên đường chéo

5 Tính Ma Trận Nghịch Đảo (inverse.py)

5.1 Thuật Toán Gauss-Jordan

Ma trận nghịch đảo được tính từ ma trận mở rộng $[A \mid I]$:

1. **Khử xuống:** Biến đổi $[A \mid I]$ thành $[U \mid X]$ (U là tam giác trên)
2. **Khử lên:** Tiếp tục khử để được $[I \mid A^{-1}]$

Điều kiện: Ma trận A phải là vuông và khả nghịch ($\det(A) \neq 0$).

5.2 Cách Sử Dụng

```

1 A = [[2, 3], [1, 5]]
2 A_inv = inverse(A) # Tra về danh sách các float
3
4 if A_inv is not None:
5     print_inverse(A) # In A và A-1
6 else:
7     print("Ma trận không khả nghịch")

```

5.3 Ứng Dụng Kiểm Thử

Kiểm chứng: $A \times A^{-1} = I$

Ví dụ:

$$\begin{pmatrix} 2 & 3 \\ 1 & 5 \end{pmatrix}^{-1} \approx \begin{pmatrix} 0.714286 & -0.428571 \\ -0.142857 & 0.285714 \end{pmatrix}$$

6 Tính Hạng và Cơ Sở (rank_basis.py)

6.1 Định Nghĩa

Với ma trận A bất kỳ:

- **Hạng (Rank):** Số lượng cột pivot = số lượng hàng độc lập tuyến tính
- **Cơ sở không gian cột (Column Space):** Các cột pivot của ma trận A gốc
- **Cơ sở không gian dòng (Row Space):** Các hàng khác 0 của ma trận dạng bậc thang
- **Cơ sở không gian nghiệm (Null Space):** Các vector cơ sở ứng với những cột không phải pivot

6.2 Thuật Toán

1. Khử ma trận A về dạng bậc thang
2. Xác định các cột pivot và hàng pivot
3. Trích xuất cơ sở từ các cột pivot
4. Tính không gian nghiệm từ các cột tự do

6.3 Cách Sử Dụng

```

1 A = [[1, 2, 3], [2, 4, 6], [0, 1, 2]]
2 rank, col_space, row_space, null_space = rank_and_basis(A)
3
4 print(f"Hạng: {rank}")
5 print(f"Cơ sở không gian cột: {col_space}")
6 print(f"Cơ sở không gian dòng: {row_space}")
7 print(f"Cơ sở không gian nghiệm: {null_space}")
8

```

```
9 # Hoac in theo dinh dang dep
10 print_rank_and_basis(A)
```

7 Kiểm Chứng và Xác Nhận (part1_demo.ipynb)

7.1 Mục Đích

Notebook `part1_demo.ipynb` cung cấp các bài kiểm thử toàn diện để xác nhận tính chính xác của các thuật toán.

7.2 Nội Dung Chính

7.2.1 Test Gaussian Elimination

- **TEST 1:** Hệ có nghiệm duy nhất (kiểm chứng bằng NumPy)
- **TEST 2:** Hệ vô nghiệm
- **TEST 3:** Hệ có vô số nghiệm
- **TEST 4:** Trường hợp đặc biệt 1×1
- **TEST 5:** Trường hợp đặc biệt 2×4 (underdetermined)
- **TEST 6:** Partial pivoting (pivot đầu = 0)

7.2.2 Test Back Substitution

- Ma trận tam giác trên với nghiệm duy nhất
- Ma trận tam giác với hệ vô nghiệm
- Ma trận tam giác với hệ vô số nghiệm
- Trường hợp đặc biệt 1×1
- Xử lý lỗi với ma trận không vuông

7.2.3 Test Determinant

- Ma trận 2×2 , 3×3 , 4×4
- Ma trận phụ thuộc tuyến tính (định thức = 0)
- Ma trận đơn vị, ma trận đường chéo

7.2.4 Test Matrix Inverse

- Ma trận 2×2 , 3×3 , 4×4
- Ma trận đơn vị
- Ma trận không khả nghịch

7.2.5 Test Rank and Basis

- Ma trận vuông khả nghịch (full rank)
- Ma trận phụ thuộc tuyến tính (rank thiếu)
- Ma trận chữ nhật (underdetermined)
- Ma trận rank = 1
- Ma trận zero (rank = 0)

7.3 Kiểm Chứng bằng NumPy và SymPy

Notebook thực hiện kiểm chứng theo đúng các nhóm TEST trong `part1_demo.ipynb`:

```

1 # TEST A: Kiem chung nghiem Gauss voi numpy.linalg.solve
2 A_test = [[2, 3], [1, 5]]
3 b_test = [7, 11]
4 my_x = gaussian_eliminate(A_test, b_test)[1]
5 np_x = np.linalg.solve(np.array(A_test, dtype=float), np.array(b_test,
6                               dtype=float))
7 flag1 = np.allclose(np.array([float(v) for v in my_x]), np_x)
8
9 # TEST B: Kiem chung dinh thuc voi numpy.linalg.det
10 A_det = [[1, 2, 3], [0, 1, 4], [5, 6, 0]]
11 my_det = float(determinant(A_det))
12 np_det = float(np.linalg.det(np.array(A_det, dtype=float)))
13 flag2 = np.isclose(my_det, np_det)
14
15 # TEST C: Kiem chung nghich dao qua A * A^(-1) ~= I
16 A_inv_test = [[2, 3], [1, 5]]
17 my_inv = inverse(A_inv_test)
18 my_inv_np = np.array([[float(v) for v in row] for row in my_inv],
19                       dtype=float)
20 A_inv_np = np.array(A_inv_test, dtype=float)
21 I_calc = A_inv_np @ my_inv_np
22 flag3 = np.allclose(I_calc, np.eye(2))
23
24 # TEST D: So sanh co so KG cot/dong/ngkiem voi SymPy
25 import sympy as sp
26
27 def ma_tran_co_so_cot(co_so, so_chieu):
28     if not co_so:
29         return np.zeros((so_chieu, 0), dtype=float)
30     return np.array(co_so, dtype=float).T
31
32 def nam_trong_tap_sinh(v, M, tol=1e-8):
33     if M.shape[1] == 0:
34         return np.linalg.norm(v) <= tol
35     he_so, *_ = np.linalg.lstsq(M, v, rcond=None)
36     return np.linalg.norm(M @ he_so - v) <= tol
37
38 def cung_khong_gian_con(co_so_1, co_so_2, so_chieu, tol=1e-8):
39     M1 = ma_tran_co_so_cot(co_so_1, so_chieu)

```

```

38     M2 = ma_tran_co_so_cot(co_so_2, so_chieu)
39
40     hang_1 = np.linalg.matrix_rank(M1, tol)
41     hang_2 = np.linalg.matrix_rank(M2, tol)
42     hang_ghep = np.linalg.matrix_rank(np.hstack([M1, M2]), tol)
43     if not (hang_1 == hang_2 == hang_ghep):
44         return False
45
46     for j in range(M1.shape[1]):
47         if not nam_trong_tap_sinh(M1[:, j], M2, tol):
48             return False
49     for j in range(M2.shape[1]):
50         if not nam_trong_tap_sinh(M2[:, j], M1, tol):
51             return False
52     return True
53
54 A_basis = [[1, 2, 3], [2, 4, 6], [0, 1, 2]]
55 my_rank, my_col_space, my_row_space, my_null_space = rank_and_basis(
56     A_basis)
57
58 A_sp = sp.Matrix(A_basis)
59 sp_col_space = [list(map(float, v)) for v in A_sp.columnspace()]
60 sp_row_space = [list(map(float, list(v))) for v in A_sp.rowspace()]
61 sp_null_space = [list(map(float, v)) for v in A_sp.nullspace()]
62
63 m = len(A_basis)
64 n = len(A_basis[0])
65 my_row_as_cols = [row[:] for row in my_row_space]
66 sp_row_as_cols = [row[:] for row in sp_row_space]
67
68 flag_col = cung_khong_gian_con(my_col_space, sp_col_space, so_chieu=m)
69 flag_row = cung_khong_gian_con(my_row_as_cols, sp_row_as_cols,
70     so_chieu=n)
71 flag_null = cung_khong_gian_con(my_null_space, sp_null_space, so_chieu
72     =n)
73
74 flag4 = flag_col and flag_row and flag_null

```

8 Hướng Dẫn Sử Dụng (README.md)

8.1 Giải Thích Hàm gaussian_eliminate

Hàm gaussian_eliminate(A, b) trả về tuple gồm 3 phần tử:

1. [0]: Ma trận dạng bậc thang (kiểu float)
2. [1]: Nghiệm hệ
3. [2]: Số lần hoán đổi hàng (int)

8.1.1 Ý Nghĩa của Nghiệm Hệ

- None: Hệ vô nghiệm

- `list[float]`: Hệ có nghiệm duy nhất
- `list[str]`: Hệ có vô số nghiệm (dạng tham số)

8.1.2 Ví Dụ Kiểm Tra Nhanh

```

1 if nghiệm_he is None:
2     print("Vo nghiệm")
3 elif len(nghiem_he) > 0 and isinstance(nghiem_he[0], str):
4     print("Vo so nghiệm:", nghiem_he)
5 else:
6     print("Nghiem duy nhat:", nghiem_he)

```

8.2 Xử Lý Sai Số Số Thực

- Dữ liệu đầu vào được chuẩn hóa về float
- Các phép kiểm tra bằng 0 sử dụng ngưỡng EPS (ví dụ: `abs(x) <= EPS`)
- Cách làm này đảm bảo tốc độ cao cho ma trận kích thước lớn mà vẫn duy trì độ chính xác trong thực nghiệm

8.3 Phân Biệt Hàm

- `gaussian_eliminate(A, b)`: Lấy dữ liệu trả về để xử lý tiếp
- `print_gaussian_eliminate(A, b)`: In kết quả ra màn hình

9 Ưu Điểm và Đặc Điểm Của Đồ Án

9.1 Hiệu Năng Và Độ Ổn Định

- Sử dụng float để tăng tốc cho ma trận kích thước lớn
- Dùng ngưỡng EPS để hạn chế nhiễu số học khi so sánh với 0
- Cân bằng tốt giữa tốc độ và độ chính xác thực nghiệm

9.2 Partial Pivoting

- Cách chọn pivot theo giá trị tuyệt đối lớn nhất
- Cải thiện ổn định số
- Giảm hiện tượng mất mát độ chính xác trong phép tính

9.3 Xử Lý Nhiều Trường Hợp

- Hệ vô nghiệm, vô số nghiệm, và nghiệm duy nhất
- Ma trận vuông, hình chữ nhật, ma trận đặc biệt
- Hàng/cột toàn 0, ma trận suy biến

9.4 Hiện Thị Rõ Ràng

- Định dạng ma trận đẹp, dễ đọc
- Hiện thị kết quả số thực gọn và nhất quán
- Đầu ra có cấu trúc và rõ ràng

10 Các Thuật Toán Không Được Sử Dụng

Theo yêu cầu của đề án, các hàm/cơ chế giải sẵn không được dùng trong việc triển khai:

- `numpy.linalg.solve`: Không dùng để giải hệ
- `numpy.linalg.inv`: Không dùng để tính ma trận nghịch đảo
- `scipy.linalg.qr`: Không dùng
- `scipy.linalg.lu`: Không dùng
- `sympy.linsolve`: Không dùng
- Echelon form/RREF từ SymPy: Không dùng

Chú ý: Các hàm thư viện trên chỉ được dùng cho **mục đích kiểm chứng** kết quả, không phải cài đặt thuật toán.

11 Kết Luận

Đề án này thành công triển khai các thuật toán cơ bản của đại số tuyến tính bằng Python:

- **Khử Gauss:** Giải hệ phương trình tuyến tính
- **Định thức:** Tính định thức thông qua khử Gauss
- **Ma trận nghịch đảo:** Sử dụng phương pháp Gauss-Jordan
- **Hạng và cơ sở:** Phân tích không gian vectơ

Tất cả các thuật toán:

1. Được triển khai từ đầu (không dùng thư viện có sẵn để tính)
2. Sử dụng `float` kết hợp `EPS` để đảm bảo ổn định số và hiệu năng
3. Xử lý đầy đủ các trường hợp đặc biệt
4. Được kiểm chứng bằng NumPy